

Communication Module Specification

Session, Transport, Link Layer, and Application Peer
for the NFC Smart Lock Protocol

Version 0.1 – Draft

July 26, 2026

Abstract

This document is the single, consolidated specification for the Lock’s communication module. It merges four previously separate documents – the Session Module, the Application Module, the Transport, and the PN532 Hardware/Link Layer binding – into one reference, organized as five parts:

- **Part I, Session Module:** Ed25519 / X25519 / HKDF-SHA256 / AES-256-GCM mutual authentication and secure messaging.
- **Part II, Application Module:** plaintext command dispatch, authorization policy, and lock actuation.
- **Part III, Transport:** APDU framing, instruction set, the ISO-DEP protocol state machine, and timing.
- **Part IV, Hardware / Link Layer:** the concrete PN532 binding of Part III’s abstract Link Layer Interface (LLI).
- **Part V, Driver Conformance Requirements:** eight deltas from a July 2026 code review, recorded here as normative requirements on the code-level driver document set that implements Part IV.

The Session and Application modules are collaborating peers, not a stack – neither is built on top of the other, and each has its own, independent external dependency (Transport for the former, the lock actuator hardware for the latter) that the other never touches directly. Transport’s own relationship to the Hardware/Link Layer, by contrast, is a genuine dependency: Transport cannot run without a conforming LLI implementation beneath it. Each part’s own scope, non-goals, and revision history are preserved from its source document; a final, document-wide revision history recording the consolidation itself appears at the very end of this document.

Contents

I	Session Module	5
1	Scope and Design Assumptions	6
1.1	Non-Goals	6
2	System Architecture	7
2.1	Components	7
2.2	Cryptographic Libraries	7

3	Long-Term Identity	7
4	Handshake Protocol	8
4.1	Rationale for the Three-Message Design	8
4.2	Message Flow	8
4.3	Message Definitions	9
4.4	Verification Rules	9
5	Shared Secret and Key Derivation	9
5.1	Shared Secret	9
5.2	Key Derivation	9
6	Secure Communication	10
6.1	Message Format	10
6.2	Nonce Strategy for Low-Volume Sessions	10
6.3	Replay Protection Within a Session	10
7	Session Termination	10
8	Security Properties	11
9	Explicitly Deferred Items (Session-Layer Scope)	12
9.1	ATECC608A Secure Element Migration	12
9.2	Relay Attack Resistance	12
10	Collaboration with the Application Module	12
11	Complete Protocol Summary	13
12	Revision History	15
II	Application Module	16
1	Scope and Layering	17
1.1	Purpose	17
1.2	Non-Goals	17
2	Relationship to Other Components	18
3	Command Set	19
3.1	Plaintext Command Envelope	19
3.2	Command Table	19
4	Authorization Model	19
5	Lock Actuation	20
5.1	Actuator Abstraction Interface (AAI)	20
5.2	Actuation Sequencing and Safety	20
6	Application-Layer Response Semantics	20

7	Collaboration with the Session Module	21
8	Security Properties	21
9	Explicitly Deferred Items	22
9.1	OTA Firmware Update Dispatch	22
9.2	Access Control List (ACL) and Roles	22
9.3	Out-of-Band BLE Command Source	23
10	Revision History	24
III	Transport	24
1	Scope and Layering	25
1.1	Purpose	25
1.2	Component Map	25
2	Link Layer Interface (LLI)	27
3	Target-Side Protocol State Machine	27
4	Data Framing and APDU Encapsulation	28
4.1	Command APDU (C-APDU): Phone → Lock	28
4.2	Response APDU (R-APDU): Lock → Phone	29
4.3	Maximum Transmission Unit and Chaining Budget	29
5	Instruction Set	29
5.1	State / Instruction Validity	30
6	Phase 1: Handshake and Key Agreement	30
6.1	Exchange 1: M1 → M2	30
6.2	Exchange 2: M3 → Authentication Status	30
7	Phase 2: Secure Communication	30
8	Timing, Liveness, and Session Guardrails	31
8.1	Handshake Timeout	31
8.2	Waiting Time Extension (Protocol-Level)	31
8.3	Link Status Polling	31
8.4	Secure-Erase Routine	31
9	Status Words (SW1/SW2) Dictionary	32
10	Conformance Requirements for a Hardware/Link Layer Binding	32
11	Revision History	33
IV	Hardware / Link Layer — PN532 Binding	33

1	Scope	34
1.1	Target Configuration	34
2	LLI Primitive Bindings	34
2.1	LLI_Activate() → TgInitAsTarget (0x8C)	34
2.2	LLI_ReceiveAPDU(timeout) → TgGetData (0x86)	35
2.2.1	Status Byte Mapping	35
2.3	LLI_SendAPDU(bytes) → TgSetData (0x8E)	35
2.3.1	Waiting Time Extension	35
2.4	LLI_GetLinkStatus() → TgGetTargetStatus (0x8A)	36
2.5	LLI_Abort() → InRelease / Re-issue TgInitAsTarget	36
3	Frame Size Conformance	36
4	Error and Status Code Reference	37
5	Revision History	37
V	Driver Conformance Requirements	37
1	lli_activate Must Not Reference a Nonexistent Command	38
2	CMD_SESSION_ABORT Must Be Honored in Every Post-Activation State	38
3	lli_config_t Field Types Must Match pn532_i2c_config_t	39
4	lli_init Must Validate gt_len/tk_len Up Front	39
5	Raw-Bypass MI-Bit Chaining: Documented Assumption, Not a Silent Gap	40
6	Hardware Reset Must Not Silently Lose SAM/Parameter Configuration	40
7	TgInitAsTarget Mode Flags Should Set PASSIVE_ONLY	41
8	Session Must Assert, Not Re-Derive, Transport's Byte Budget	41
9	Summary Table	41
10	Revision History	42
VI	Document Set History	42
1	Revision History (This Consolidated Document)	43

Part I Session Module

Part Overview

This document specifies the cryptographic session module of the NFC smart lock protocol: mutual device authentication and secure-channel establishment between a mobile application (“Phone”) and an ESP32-based smart lock controller (“Lock”). It combines long-term Ed25519 identities, ephemeral X25519 key agreement, a three-message SIGMA-style handshake, HKDF-SHA256 key derivation, and AES-256-GCM authenticated encryption to provide mutual authentication, confidentiality, integrity, and forward secrecy for each session. This module has no knowledge of command semantics, authorization policy, or lock actuation – that half of the task belongs to a separate, *equal-standing* module, the Application Module (Part II), with which this document collaborates directly (Section 10). Neither module is a sub-component of the other; each depends on the other for a different half of the same task, and neither is functional alone. This module also makes use of a third, independent component – the Transport (Part III) – to physically move its handshake and ciphertext bytes, but it does not control, configure, or own the transport’s state machine or timing; from this module’s point of view, the transport is simply an opaque carrier that it hands bytes to and receives bytes from, in the same way the application module hands this module plaintext and receives plaintext back. This revision splits command dispatch, authorization, and actuation out of the document’s own predecessor (a single combined document, historically also named *smartlock_application_layer.pdf* v1.0) into the Application Module (Part II); see Section 12 for details. No cryptographic behavior changes are introduced by this split.

1 Scope and Design Assumptions

This specification governs cryptographic session establishment and secure messaging between the Phone and the Lock. The following scope decisions are in effect for this revision and are treated as explicit, reviewed trade-offs rather than oversights.

- **Handshake structure:** Three-message mutual authentication (upgraded from the earlier two-message design) to eliminate signature ambiguity. See Section 4.1.
- **Secure element:** The ESP32 currently stores the long-term Ed25519 private key in software (libsodium, RAM/flash). Migration to an ATECC608A secure element is planned; see Section 9.1.
- **Session message volume:** Each NFC session is expected to carry a low number of application messages (tens, not thousands). This justifies the use of randomly generated AES-GCM nonces without a counter construction; see Section 6.2.
- **Relay attacks:** Out of scope for this revision. This layer does not attempt to bound proximity or handshake latency; see Section 9.2.

1.1 Non-Goals

This specification does not cover: provisioning workflows (manufacturing, QR-code pairing, onboarding UX), firmware update security, physical tamper response, power analysis / side-channel

resistance of the current software Ed25519 implementation, transport framing (APDU encapsulation, instruction set, ISO-DEP timing – see the Transport (Part III)), or anything that happens to plaintext bytes once they are handed to the application module, including command dispatch, authorization policy, and lock actuation (see the Application Module (Part II), a collaborating peer of this document, not a downstream consumer of it). This module authenticates *devices*; it does not authorize *actions* – that determination belongs entirely to the application module.

2 System Architecture

2.1 Components

Component	Platform	Role (this layer)
Mobile Application	Flutter (iOS/Android)	Initiates NFC session, performs the handshake, encrypts/decrypts application payloads. Command content is opaque to this layer – see the Application Module (Part II).
Lock Controller	ESP32 (ESP-IDF)	Verifies Phone identity, performs the handshake, encrypts/decrypts application payloads. Does not interpret plaintext content or actuate hardware – see the Application Module (Part II).

2.2 Cryptographic Libraries

Platform	Library	Responsibility
Flutter	<code>cryptography</code>	Ed25519 sign/verify, X25519 agreement, HKDF-SHA256, AES-256-GCM, secure RNG
Flutter	<code>flutter_secure_storage</code>	Storage of the Ed25519 private identity key (Keychain / Keystore)
Flutter	<code>flutter_nfc_kit</code> / <code>nfc_manager</code>	NFC transport only – no cryptographic operations; owned by the transport layer
ESP32	<code>libsodium</code>	Ed25519 sign/verify, X25519 agreement, secure RNG; software key storage today, ATECC608A-backed in a future revision
ESP32	<code>mbedtls</code> (ESP-IDF)	HKDF-SHA256, AES-256-GCM

No custom cryptographic primitives are implemented on either platform.

3 Long-Term Identity

Each device holds a permanent Ed25519 identity key pair, generated once at provisioning time.

Phone	Ed25519 Private Key (Keychain / Keystore) Ed25519 Public Key (provisioned to Lock)
Lock	Ed25519 Private Key (libsodium; ATECC608A in a future revision) Ed25519 Public Key (provisioned to Phone / backend)

Public key distribution (manufacturing, QR-code pairing, onboarding) is governed by the trust model and is outside the scope of this document. This specification assumes both parties already hold an authentic copy of the peer's Ed25519 public key before the handshake begins.

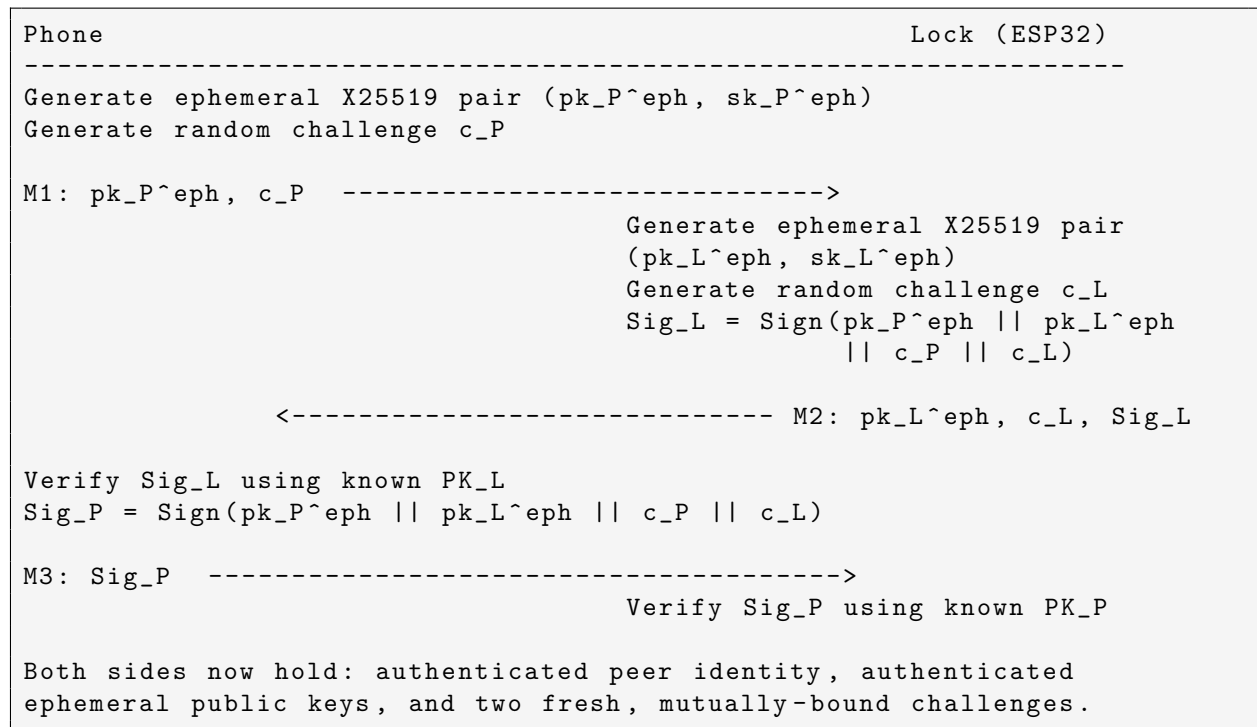
4 Handshake Protocol

4.1 Rationale for the Three-Message Design

The original two-message design had the Phone sign “the ephemeral public key and both challenges,” but the Phone's first message is sent before it has seen the Lock's challenge. A signature that does not cover the Lock's challenge is not bound to the Lock's contribution to the session, which permits splicing a valid Phone signature from one session into another session carrying a different Lock challenge.

This revision adopts a SIGMA-style three-message handshake: the Lock speaks after the Phone and signs both parties' values, and the Phone closes the handshake with a final signature that also covers both parties' values. Every signature produced by either party covers both ephemeral public keys and both challenges, so no signature is valid outside the session it was created for.

4.2 Message Flow



4.3 Message Definitions

Msg	Sender	Contents
M1	Phone	pk_P^{eph} (X25519, 32 bytes), c_P (random challenge, 32 bytes)
M2	Lock	pk_L^{eph} (X25519, 32 bytes), c_L (random challenge, 32 bytes), $Sig_L(pk_P^{eph} pk_L^{eph} c_P c_L)$ (Ed25519, 64 bytes)
M3	Phone	$Sig_P(pk_P^{eph} pk_L^{eph} c_P c_L)$ (Ed25519, 64 bytes)

Concatenation order ($pk_P^{eph} || pk_L^{eph} || c_P || c_L$) is fixed and identical for both signatures, and must be implemented identically on both platforms. A protocol version byte and a domain separation prefix (e.g. "SLOCK-HS-v1") SHOULD be prepended to the signed payload on both sides to prevent cross-protocol signature reuse.

4.4 Verification Rules

1. Upon receiving M2, the Phone MUST verify Sig_L using the Lock's provisioned long-term public key PK_L before proceeding to M3. On failure, the Phone MUST abort the session and MUST NOT derive any session key material.
2. Upon receiving M3, the Lock MUST verify Sig_P using the Phone's provisioned long-term public key PK_P before treating the session as authenticated. On failure, the Lock MUST abort; this layer MUST NOT signal a successful handshake to the application layer for this session.
3. Both parties MUST reject a handshake if pk_P^{eph} or pk_L^{eph} is the all-zero point or otherwise fails X25519 validity checks (contributory behavior check / small-subgroup rejection, as provided by libsodium and the `cryptography` package by default).
4. Challenges c_P , c_L MUST be generated with a CSPRNG and MUST NOT be reused across sessions.

5 Shared Secret and Key Derivation

5.1 Shared Secret

Both devices independently compute the X25519 shared secret:

```
SharedSecret = X25519(local_ephemeral_private, remote_ephemeral_public)
```

No key material is transmitted during this step; each side derives the same value from its own ephemeral private key and the peer's ephemeral public key received during the handshake.

5.2 Key Derivation

The session key is derived via HKDF-SHA256, binding in both challenges so that the resulting key is cryptographically tied to this specific handshake instance:

```
PRK = HKDF-Extract(salt = c_P || c_L, ikm = SharedSecret)
Key = HKDF-Expand(PRK, info = "phone->esp" | "esp->phone", L = 32)
```

Two directional keys are derived using distinct info strings to prevent key reuse across communication directions:

Direction	HKDF info string
Phone → Lock	"phone->esp"
Lock → Phone	"esp->phone"

Each derived key is 32 bytes (256 bits), used directly as an AES-256-GCM key.

6 Secure Communication

6.1 Message Format

All payloads exchanged after the handshake are encrypted with AES-256-GCM using the appropriate directional key. This layer treats the plaintext as an opaque byte string; its structure and interpretation are defined by the Application Module (Part II).

Field	Size	Description
Nonce	12 bytes	CSPRNG-generated, unique per message per key (see Section 6.2)
Ciphertext	variable	AES-256-GCM output for the application-layer plaintext
Auth Tag	16 bytes	GCM authentication tag

6.2 Nonce Strategy for Low-Volume Sessions

Given the low expected number of messages per session (typically a single unlock command and acknowledgment, rarely exceeding a few dozen messages), random 96-bit nonces are considered acceptable for this revision. The birthday-bound collision risk for random GCM nonces becomes significant only as message counts approach the range of 2^{32} under a single key; the per-session ephemeral key lifetime and low message volume keep the collision probability negligible.

Recommendation for future hardening: should session message volume increase substantially (e.g. streaming telemetry, high-frequency polling), this SHOULD be revisited in favor of a deterministic counter-based nonce (optionally salted with a random per-session prefix) to remove reliance on RNG quality for nonce uniqueness.

6.3 Replay Protection Within a Session

Because a new session key is derived per NFC session, and each session is short-lived and single-purpose (e.g. one unlock command), sequence-number based replay protection within a session is currently OPTIONAL. If a session is extended to carry multiple sensitive commands, an incrementing sequence number SHOULD be included in the associated data (AD) passed to AES-GCM. Whether a given plaintext command is safe to replay within a session (e.g. an idempotent status query vs. a state-changing unlock command) is an application-layer policy decision – see the Application Module (Part II).

7 Session Termination

Upon completion or abort of the NFC session, both parties MUST securely erase the following from memory:

- Ephemeral X25519 private key (sk^{eph})
- X25519 shared secret
- HKDF intermediate output (PRK)
- Derived AES-256 session keys (both directions)

Only the long-term Ed25519 identity key persists between sessions. Each subsequent NFC interaction executes the full three-message handshake with freshly generated ephemeral keys and challenges; no session state is resumed or cached. This erase routine is invoked by several trigger conditions defined at the transport layer (session abort, handshake timeout, link loss, frame-integrity error, signature or auth-tag failure); see the Transport (Part III) §6.

8 Security Properties

Property	Mechanism	Notes
Mutual Device Authentication	Ed25519 signatures over transcript ($pk_P^{eph}, pk_L^{eph}, c_P, c_L$)	Both signatures cover the full transcript, closing the M1-only binding gap of the prior two-message design
Confidentiality	AES-256-GCM	Directional keys via HKDF info separation
Integrity	AES-GCM authentication tag	16-byte tag per message
Key Agreement	X25519	Ephemeral, per-session
Key Derivation	HKDF-SHA256	Challenges bound into salt
Replay Protection (handshake)	Fresh challenges in signed transcript	Prevents cross-session signature reuse
Forward Secrecy	Fresh X25519 pair per session, destroyed at termination	Compromise of long-term key does not expose past session traffic
Long-Term Key Protection (Phone)	OS Keychain / Keystore via <code>flutter_secure_storage</code>	
Long-Term Key Protection (Lock)	Software (libsodium) in this revision; hardware secure element planned	See Section 9.1
Relay Attack Resistance	Not yet implemented – see below	Out of scope this revision; see Section 9.2

Note: device authentication (this table) is a necessary but not sufficient condition for authorizing an action such as unlocking. Whether an authenticated device is *permitted* to issue a given command is defined entirely by the Application Module (Part II).

9 Explicitly Deferred Items (Session-Layer Scope)

The following items are acknowledged gaps, intentionally deferred rather than overlooked, and should be tracked for future revisions of this document. Authorization-model and lock-actuation gaps, previously listed here, now live in the Application Module (Part II) (see that document’s own deferred-items section); this section retains only cryptographic and handshake-level gaps.

9.1 ATECC608A Secure Element Migration

The ESP32 currently manages its long-term Ed25519 private key in software via libsodium. A future revision will migrate long-term key storage and signing operations to the Microchip ATECC608A secure element, which provides:

- Hardware-isolated private key storage (key never leaves the secure element)
- Hardware-accelerated ECDSA (note: ATECC608A natively supports P-256 ECDSA, not Ed25519 – migration will require either (a) switching the long-term identity scheme to ECDSA P-256, or (b) using the ATECC608A purely as a protected key vault / HMAC-based challenge engine while retaining software Ed25519, depending on final part variant and firmware support)
- Tamper and side-channel resistant signing
- A hardware root of trust for device provisioning

Action item: confirm target ATECC608A variant and its supported curve set before finalizing whether the long-term identity scheme remains Ed25519 or migrates to P-256, as this affects both firmware and the mobile verification code path.

9.2 Relay Attack Resistance

This revision does not defend against relay attacks, in which an attacker relays the NFC exchange between a legitimate Phone and a legitimate Lock over a long-distance channel. Candidate mitigations for future revisions include:

- Distance-bounding / round-trip time (RTT) measurement with a strict timeout on the M1 → M2 → M3 exchange, layered on top of the transport layer’s existing handshake timeout T_{HS} (see the Transport (Part III) §5.1)
- Leveraging NFC’s inherent short range together with a hard timeout as a first-order mitigation (does not fully prevent relay but raises attack cost)

10 Collaboration with the Application Module

The session and application modules are peers that jointly implement one task – an authenticated, confidential command exchange – and neither half is meaningful on its own: a Lock that only ran this module could authenticate a Phone and open a secure channel to it, but could never act on anything sent over that channel; a Lock that only ran the application module would have commands to dispatch and hardware to drive, but no way to know whether the byte string requesting them came from a trusted device. Figure 1 shows this as two side-by-side components rather than a stack.

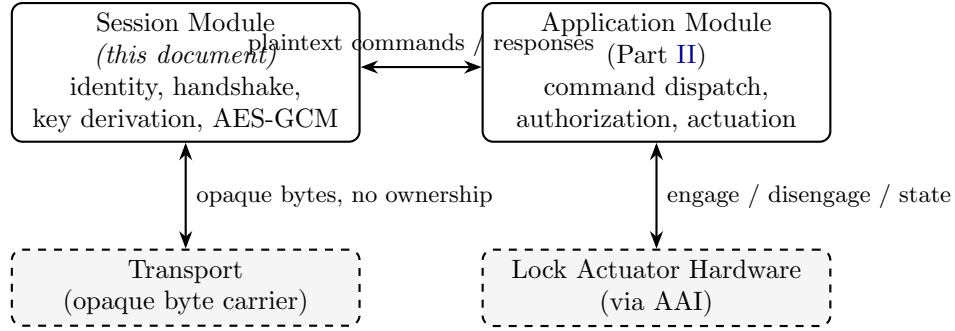


Figure 1: The session and application modules collaborate as peers. Each also has its own, independent external dependency (transport for this module, actuator hardware for the application module) that the other module never touches directly.

- **Session Module → Application Module:** once M3 is verified and the session transitions to *Secure Session*, every successfully decrypted and authenticated plaintext byte string is delivered to the application module unmodified. This module does not parse, validate, or otherwise interpret plaintext content – that is the application module’s job, not a job this module delegates from a position above it.
- **Application Module → Session Module:** the application module supplies a plaintext byte string to be encrypted with the appropriate directional key and a fresh nonce; this module returns the nonce, ciphertext, and auth tag for the transport to carry. This module has no visibility into why the application module chose that plaintext.
- **Failure signaling:** an AES-GCM authentication tag mismatch is a failure local to this module and MUST NOT be forwarded to the application module as a plaintext message – it triggers the secure erase routine (Section 10) and session termination directly.
- This module holds no notion of “commands,” “authorization,” or “actuation.” Those are defined entirely in the Application Module (Part II), and this module has no more authority over that document’s decisions than it does over the reverse.
- Symmetrically, this module owns the transport relationship shown in Figure 1: the transport (see the Transport (Part III)) carries this module’s bytes without needing to understand them, but this module does not configure, drive, or make decisions on behalf of the transport’s own state machine or timing – that remains entirely the transport document’s concern.

11 Complete Protocol Summary

Phone	Lock (ESP32)
Load Ed25519 identity key	Load Ed25519 identity key
Generate X25519 ephemeral pair (pk_P, sk_P)	
Generate challenge c_P	

```

M1: pk_P, c_P ----->
                                Generate X25519 ephemeral pair
                                (pk_L, sk_L)
                                Generate challenge c_L
                                Sig_L = Sign(pk_P||pk_L||c_P||c_L)

                                <----- M2: pk_L, c_L, Sig_L

Verify Sig_L using PK_L
Sig_P = Sign(pk_P||pk_L||c_P||c_L)

M3: Sig_P ----->
                                Verify Sig_P using PK_P

[Both sides] SharedSecret = X25519(local_sk, remote_pk)
[Both sides] PRK      = HKDF-Extract(salt = c_P||c_L, ikm = SharedSecret)
[Both sides] K_p2e    = HKDF-Expand(PRK, "phone->esp", 32)
[Both sides] K_e2p    = HKDF-Expand(PRK, "esp->phone", 32)

<===== AES-256-GCM encrypted payloads, exchanged with the =====>
<== application module (Part III: Application Module) as a peer ==>

[Both sides] Destroy: sk_eph, SharedSecret, PRK, K_p2e, K_e2p

```

12 Revision History

Version	Change	Date
0.1	Initial two-message handshake draft	—
0.2	Upgraded to three-message SIGMA-style handshake to close signature transcript binding gap; documented ATECC608A migration plan; scoped authorization and relay-attack mitigation as deferred work. Published as a single combined document (historically named <i>smartlock_application_layer.pdf</i>), combining cryptographic session-establishment content with command dispatch, authorization, and lock-actuation content. (Originally labeled “1.0”; renumbered under 0.3 below to match this document set’s sub-1.0 versioning convention.)	July 2, 2026
0.3	Split out non-cryptographic scope, and renumbered. Command dispatch, authorization model, and lock actuation logic (formerly §9.2 of the previous revision) moved to a new sibling document, the Application Module (Part II) (v0.1). This document renamed to the Session Module (Part I) and its scope narrowed to cryptographic session establishment and secure messaging only, aligning with the component map already published in the Transport (Part III) §1.2. Added a data-contract section formalizing the interface with the new application layer document. Version numbering corrected from “1.1” to follow the project-wide convention that no document in this set reaches 1.0 before a stable freeze (see the numbering note below). No cryptographic behavior changes.	July 24, 2026
0.4	Reframed as a peer module, not a stacked layer. Former “Interface to the Application Layer” section (with “upward”/ “downward” language) replaced by §10, “Collaboration with the Application Module,” which presents the session and application modules as equal-standing collaborators (Figure 1) rather than one sitting on top of the other. Clarified throughout that this module does not own, configure, or control the transport’s state machine – the transport is an independent, opaque byte carrier used by this module, exactly as this module is used by the application module. No cryptographic or behavioral changes.	July 26, 2026

Note on numbering: this project has not yet reached a stable v1.0 release. Version numbers below 1.0 are used throughout the document set (session, application, transport, hardware/link layer, LLI) until the first frozen release, and are not directly comparable across documents – e.g. this document’s

0.4 does not imply it is “ahead of” or “behind” application layer 0.2 in maturity, only that it is this document’s fourth drafted revision. An earlier draft of this document briefly used a 1.x numbering scheme (1.0–1.2); that was inconsistent with the convention followed by the rest of the document set and has been corrected above.

Part II Application Module

Part Overview

This document specifies the application module of the NFC smart lock protocol: the plaintext command set exchanged once a secure session is established, the current authorization policy, and the logic that actuates the physical lock mechanism. This module works as an *equal-standing peer* of the cryptographic session module (the Session Module (Part I)), not as a layer built on top of it – the two collaborate directly (Section 2), and neither is functional without the other. This module has no knowledge of cryptographic operations, key material, or transport/APDU framing (the Transport (Part III)); it receives only plaintext byte strings that the session module has already authenticated as originating from a device holding a provisioned Ed25519 identity, and it returns plaintext byte strings for the session module to encrypt. Unlike a pure communications layer, however, this module also owns a second, independent responsibility that has nothing to do with messaging at all: driving the physical lock actuator (Section 5). This Part was split out of a single combined predecessor document (historically named *smartlock_application_layer.pdf* v1.0, later renumbered 0.2 and renamed into the Session Module, Part I – see that part’s revision history), whose cryptographic content now lives in the Session Module (Part I); see Section 10. It is deliberately scoped narrowly today so that upcoming work – OTA firmware update dispatch, an access control list (ACL), and an out-of-band BLE command source – can be added as clearly bounded extensions (Section 9) rather than accreting into a single monolithic module again.

1 Scope and Layering

1.1 Purpose

This layer owns three responsibilities, and only these three:

1. **Command dispatch** – interpreting the plaintext byte string delivered by the session layer as a structured command and routing it to the correct handler (Section 3).
2. **Authorization** – deciding whether an authenticated device is permitted to have a given command executed (Section 4).
3. **Actuation** – driving the physical lock mechanism and reporting its resulting state (Section 5).

It MUST NOT implement, re-implement, or depend on the internal details of Ed25519/X25519/HKDF/AES-GCM operations, nonce generation, or key storage; those are exclusively the session module’s responsibility. It MUST NOT assume anything about how its plaintext arrived (NFC, or in a future revision, BLE – see Section 9.3) beyond the guarantee, provided by whichever module delivered it, that the sender’s long-term identity has been authenticated for this session. Note that this module is not purely a messaging consumer: Section 5 gives it a second external dependency, the physical actuator, that has no counterpart in the session module and does not route through it.

1.2 Non-Goals

This specification does not cover: cryptographic session establishment or secure messaging (the Session Module (Part I)), APDU framing, instruction sets, or ISO-DEP timing (the Transport

(Part III)), NFC controller specifics (the Hardware/Link Layer (Part IV)), provisioning workflows, or physical/mechanical design of the lock hardware itself (motor, solenoid, or deadbolt selection). Command sources other than the NFC secure session – in particular the planned BLE out-of-band channel – are explicitly deferred; see Section 9.3.

2 Relationship to Other Components

This module is not a rung on a protocol ladder. It has exactly two external dependencies, of two fundamentally different kinds, and neither is “below” it:

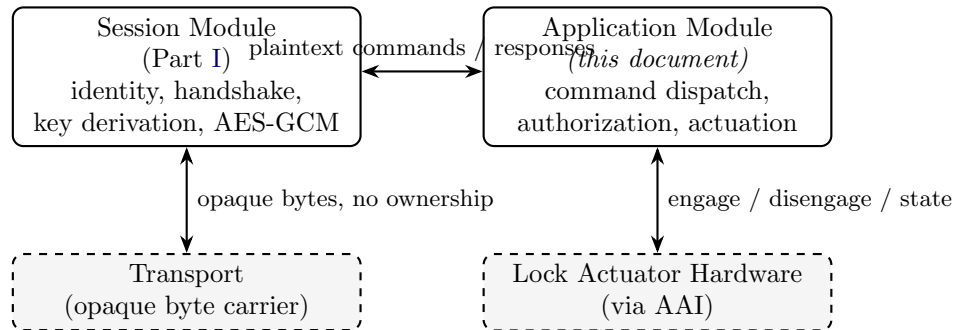


Figure 2: This module collaborates with the session module as a peer, not a downstream consumer. It also depends directly on the lock actuator hardware – a dependency the session module has no part in, and no visibility into.

- **The session module** (Ed25519 / X25519 / HKDF / AES-GCM handshake and secure messaging, the Session Module (Part I)) hands this module authenticated plaintext and accepts plaintext back for encryption. It has no knowledge of command content, and this module has no knowledge of how that plaintext was authenticated or encrypted. Section 7 defines this collaboration in full; it is bidirectional, not a call from a higher layer into a lower one.
- **The lock actuator hardware** (Section 5) is this module’s other, unrelated dependency: physical GPIO/motor control that the session module never touches and has no reason to know exists. This is precisely why the two are peers rather than a stack – the application module’s job is not simply “the layer that consumes session-module output”; roughly half of what it does (Section 5) has nothing to do with communications at all.
- **Transport** (APDU framing, instruction set, ISO-DEP timing; the Transport (Part III)) and the **hardware/link layer** (concrete NFC controller driver; the Hardware/Link Layer (Part IV)) are real dependencies of the session module, not of this one. This module never sees an APDU, a nonce, or a ciphertext byte; those stay entirely within the session module’s side of Figure 2. The session module, in turn, does not own or control the transport’s state machine or timing – it only hands the transport opaque bytes, the same way this module hands the session module opaque plaintext.

A device is authenticated by the session module before any plaintext ever reaches this document’s dispatch logic (Section 3). This module never sees, and cannot be tricked by, an unauthenticated message – but that guarantee comes from a peer collaborator, not from a layer underneath it.

3 Command Set

3.1 Plaintext Command Envelope

The plaintext byte string delivered by the session layer (the decrypted content of a `CMD_SECURE_PAYLOAD` exchange) is structured as follows:

Field	Size	Description
OPCODE	1 byte	Identifies the command; see Table 2
ARGS	variable	Opcode-specific arguments, may be zero-length

The corresponding response, returned to the session layer for encryption, uses the same shape with OPCODE replaced by a status byte (Section 6).

3.2 Command Table

Command	OPCODE	Description
<code>CMD_UNLOCK</code>	0x01	Requests actuation of the lock mechanism to the unlocked position. ARGS: none (this revision).
<code>CMD_LOCK</code>	0x02	Requests actuation of the lock mechanism to the locked position. ARGS: none (this revision).
<code>CMD_STATUS</code>	0x03	Requests the current actuator state without changing it. ARGS: none.

Table 2: Application-layer opcodes defined in this revision.

An OPCODE not listed in Table 2 MUST be rejected with `APP_STATUS_UNKNOWN_CMD` (Section 6) and MUST NOT reach the actuation logic in Section 5. New commands (e.g. for OTA dispatch, Section 9.1) are added by extending Table 2, not by modifying the envelope format.

4 Authorization Model

This revision’s authorization policy is intentionally simple, and is stated here explicitly rather than left implicit:

Authentication currently implies authorization. Any device that successfully completes the session-layer handshake (i.e. proves possession of a private key corresponding to a provisioned Ed25519 public key) is permitted to issue any command in Table 2.

There is no per-device role, scope, or permission check in this revision: this layer does not distinguish an owner’s phone from a guest’s phone, or an unlock request from a status query, for authorization purposes – both are governed by the same rule above. This is a conscious, reviewed scope limitation carried forward from the prior combined specification, not an oversight. Introducing differentiated authorization (an ACL, roles, or time-limited credentials) is deferred work; see Section 9.2.

5 Lock Actuation

5.1 Actuator Abstraction Interface (AAI)

To keep this document independent of the specific motor/solenoid driver circuit, actuation is expressed against a small abstract interface, mirroring the pattern used for the transport layer’s Link Layer Interface (LLI). Any conforming actuator driver **MUST** implement:

Primitive	Required Behavior
<code>AAI_Engage()</code>	Drives the mechanism to the unlocked position. Returns success/failure once motion completes or a hardware time-out elapses.
<code>AAI_Disengage()</code>	Drives the mechanism to the locked position. Same completion/timeout semantics as <code>AAI_Engage()</code> .
<code>AAI_GetState()</code>	Returns the current mechanical state: <code>LOCKED</code> , <code>UNLOCKED</code> , or <code>UNKNOWN</code> (e.g. no position sensor fitted, or sensor fault).

The concrete driver behind the AAI (GPIO assignment, motor driver IC, timing) is out of scope for this document, matching the treatment of the LLI in the Transport (Part III).

5.2 Actuation Sequencing and Safety

1. `CMD_UNLOCK` / `CMD_LOCK` handling **MUST** call Section 4’s authorization check before calling `AAI_Engage()` / `AAI_Disengage()`. There is currently no case in which the check can fail for an authenticated device (see Section 4), but the call **MUST** remain in place so that future ACL enforcement (Section 9.2) has a single insertion point.
2. Immediately before actuation, this layer **SHOULD** confirm (via whatever session-liveness signal the transport layer exposes) that the session is still active, to avoid actuating on behalf of a session that has already been torn down.
3. `AAI_GetState()` **SHOULD** be called both before and after actuation so the response to the Phone (Section 6) reflects the mechanism’s actual resulting state rather than an assumption that the drive command succeeded.
4. This revision does not define behavior for a mechanism that reports `UNKNOWN` state after actuation beyond returning `APP_STATUS_ACTUATOR_FAULT`; retry policy is left to a future revision.

6 Application-Layer Response Semantics

Transport-layer status words (`SW1/SW2`, defined in the Transport (Part III) §8) only describe conditions visible to the transport and session layers (e.g. malformed APDU, handshake failure, auth-tag mismatch). They cannot describe application-layer outcomes, because by the time a status word is set the transport layer has no visibility into plaintext content. This layer therefore defines its own status byte, carried as the first byte of the encrypted plaintext response described in Section 3:

Status	Value	Meaning
APP_STATUS_OK	0x00	Command executed (or, for CMD_STATUS, state read) successfully; actuator state follows in ARGS
APP_STATUS_UNKNOWN_CMD	0x01	OPCODE not present in Table 2
APP_STATUS_ACTUATOR_FAULT	0x02	AAI reported UNKNOWN state after an actuation attempt

A transport- or session-layer failure (e.g. auth-tag mismatch, signature failure) never reaches this layer at all and therefore never produces one of these status bytes – those failures are signaled entirely below this layer, per the Transport (Part III) §8 and the Session Module (Part I) §10.

7 Collaboration with the Session Module

This is the same collaboration described from the other side in the Session Module (Part I) §10; it is repeated here so this document is self-contained, not because the two directions are owned by different parties.

- **Session Module → Application Module:** this module receives only plaintext byte strings that the session module has already decrypted and authenticated. It MUST treat the sender’s long-term identity as authenticated for every message it receives; it MUST NOT re-derive or re-check any cryptographic property – that would duplicate, not respect, the session module’s responsibility.
- **Application Module → Session Module:** this module returns a plaintext byte string (status byte plus ARGS, Section 6) for the session module to encrypt with the appropriate directional key. This module never has access to, and never handles, key material, nonces, or ciphertext directly – not because it sits below the session module, but because that is simply not this module’s job.
- **No independent replay protection:** this module relies entirely on the session module’s per-session key freshness (the Session Module (Part I) §6.3) for replay resistance. It does not maintain its own sequence numbers in this revision.

8 Security Properties

Property	Mechanism	Notes
Authorization	Implicit – authentication equals authorization	Any authenticated identity is currently authorized for any command in Table 2; see Section 4
Actuation Integrity	Single-writer access to the AAI from the application task	No concurrent-actuation protection is defined beyond the transport layer’s single-session-at-a-time model

Property	Mechanism	Notes
Command Replay	Inherited from session layer key freshness	This layer adds no independent nonce or sequence number (this revision)
Command Injection	Strict OPCODE allow-list (Table 2)	Unknown opcodes rejected before reaching Section 5

9 Explicitly Deferred Items

The following items are acknowledged gaps, intentionally deferred rather than overlooked. Each is scoped here as a bounded extension point so that implementing it does not require re-splitting this document later – if any one of these grows large enough to warrant it, it **SHOULD** become its own sibling specification (following the same layering discipline used to separate this document from the Session Module (Part I)) rather than being folded back into this file.

9.1 OTA Firmware Update Dispatch

Planned as one or more new opcodes in Table 2 (e.g. `CMD_OTA_BEGIN`, `CMD_OTA_CHUNK`, `CMD_OTA_COMMIT`). Open questions for a future revision:

- Whether OTA payloads route through the existing 227-byte unchained plaintext budget (the Transport (Part III) §4.3) or require the transport layer to implement chaining, which is explicitly out of scope for that document’s current revision.
- Firmware image authenticity (signing scheme, and whether it reuses the long-term Ed25519 identity key or a separate release key) – this is a session-layer or provisioning-layer concern and **MUST NOT** be designed inside this document.
- Whether OTA commands require a stricter authorization check than Section 4’s current any-authenticated-device rule, which strongly motivates completing Section 9.2 first.

9.2 Access Control List (ACL) and Roles

Planned replacement for Section 4’s current policy:

- An access control list of authorized Phone public keys per Lock, or a signed capability/credential issued by a backend authority.
- A revocation mechanism (e.g. denylist, short-lived signed credentials, or an online revocation check) so a lost or decommissioned phone can be removed without re-provisioning the Lock’s entire trust store.
- Optional role/scope differentiation (e.g. owner vs. guest vs. time-limited access), which would extend Section 4’s check to be per-command rather than all-or-nothing, and would give Section 5’s insertion point (item 1) its first real failure case.

9.3 Out-of-Band BLE Command Source

Planned addition of BLE as a second transport carrying commands into this layer, alongside NFC. Open questions for a future revision:

- Whether BLE sessions reuse the existing session-layer handshake unchanged (preferred, since this layer’s authorization model already only depends on “authenticated identity,” not on which transport carried it), or require a BLE-specific pairing/bonding step layered underneath it.
- If reused, this document’s Section 1.1 contract (“has no knowledge of transport”) already accommodates a second transport with no changes required here – the work belongs in a new a future *BLE Transport* sibling document sibling to the Transport (Part III), not in this document.
- Whether any command in Table 2 should be restricted to one transport only (e.g. disallowing CMD.UNLOCK over a longer-range BLE link for relay-risk reasons) is an authorization-policy question and belongs in Section 9.2, not in a transport document.

10 Revision History

Version	Change	Date
0.1	Initial standalone revision. Split out of a single combined predecessor document (historically named <i>smartlock_application_layer.pdf</i> v1.0; later renumbered 0.2 within the Session Module (Part I)’s history), which previously mixed cryptographic session-establishment content (now the Session Module (Part I) v0.3) with command dispatch, authorization, and lock-actuation content. Authorization Model content carried forward from that document’s §9.2 essentially unchanged in substance. Newly defined in this revision: the plaintext command envelope and opcode table (Section 3), the Actuator Abstraction Interface (Section 5), and application-layer response status codes (Section 6), none of which were specified in the prior combined document. Deferred-items section restructured around three named extension points (OTA, ACL, BLE) to bound future growth of this document.	July 24, 2026
0.2	Reframed as a peer module, not a stacked layer. Former “Relationship to Other Layers” and “Interface to the Session Layer” sections (which described this module as sitting “directly above” the session module and used “upward”/“downward” language) replaced by §2 and §7, which present the session and application modules as equal-standing collaborators (Figure 2). Made explicit that this module’s actuator hardware dependency (Section 5) is independent of, and parallel to, its session-module dependency – this module is not solely a communications consumer. Clarified that the session module does not own or control transport on this module’s behalf. No functional changes to commands, authorization, or actuation logic.	July 26, 2026

Part III Transport

Part Overview

This document specifies the transport that carries the cryptographic handshake and secure-messaging bytes produced by the the Session Module (Part I) over any ISO/IEC 14443-4 (ISO-DEP) compliant contactless link. It is intentionally **hardware-agnostic**: it defines APDU framing, the instruction set, the target-side protocol state machine, timing requirements, and status codes purely in terms of the ISO/IEC 14443-4 standard and ISO/IEC 7816-4 APDU conventions. It does *not* reference any specific NFC controller chip, command set, or register. Binding this specification to a concrete controller (e.g. PN532, ST25R3911B, CR95HF) is the responsibility of a separate *Hardware/Link Layer* document, which implements the abstract interface defined in Section 2 for that controller – a genuine dependency relationship, since this document cannot run without a conforming LLI implementation underneath it. This document’s relationship to the session module (Section 1) is of a different kind: this document carries the session module’s bytes without inspecting or owning them, and neither document sits above the other. This document also has no dependency at all on the application module (the Application Module (Part II)), which collaborates with the session module directly, as an equal-standing peer, rather than through this document.

1 Scope and Layering

1.1 Purpose

This specification is deliberately independent of any single NFC front-end IC. The Lock’s protocol-handling logic (handshake state machine, APDU parsing, status word generation – all defined in this document, not to be confused with the *Application Module*’s command-handling logic; see Section 1.2) should compile and run unchanged against any controller that can act as an ISO/IEC 14443-4 Type A PICC (Target) at 106 kbps, provided that controller’s driver implements the **Link Layer Interface (LLI)** defined in Section 2.

1.2 Component Map

This document’s own dependency chain is a genuine stack: it cannot function without a conforming Hardware/Link Layer beneath it. Its relationship to the session module is different in kind: it carries the session module’s bytes as an opaque payload, but does not sit above or below it. The session and application modules, in turn, are not stacked relative to each other either. Figure 3 makes this explicit rather than leaving it to a table that reads top-to-bottom.

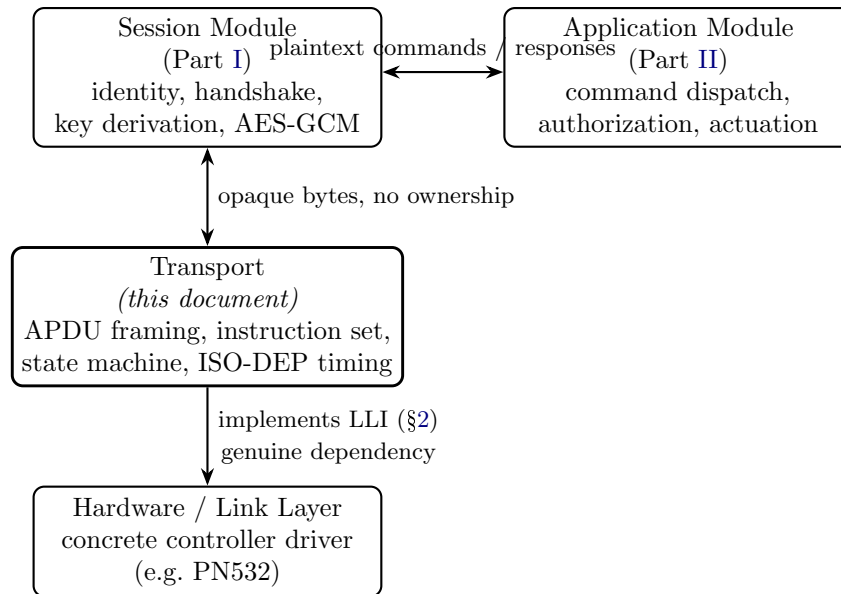


Figure 3: This document (bold border) genuinely depends on a conforming Hardware/Link Layer beneath it – that relationship is a real stack. Its relationship with the session module is not: it carries the session module’s bytes without inspecting or controlling them, exactly as the session module carries the application module’s plaintext without inspecting or controlling it. The session and application modules are peers, collaborating directly with each other; neither routes through this document to reach the other.

Component	Responsibility
Session Module	Ed25519 / X25519 / HKDF / AES-GCM handshake and secure messaging. Defined in the Session Module (Part I). Operates on opaque byte payloads; has no knowledge of NFC, APDUs, or RF. Collaborates directly with the application module as a peer – not through this document.
Application Module	Command dispatch (CMD_UNLOCK), authorization policy, lock actuation logic. Defined in the Application Module (Part II). Receives plaintext from the session module; has no knowledge of cryptography or of this document’s existence.
Transport (this document)	APDU framing, instruction set, protocol-level state machine, ISO-DEP timing (FWT/WTX budget), status word dictionary, chaining policy. Carries the session module’s bytes as an opaque payload; depends on ISO/IEC 14443-4 and ISO/IEC 7816-4 semantics via the LLI, which is a real, load-bearing dependency on the layer below.
Hardware / Link Layer	Concrete controller driver (command opcodes, register access, bus protocol – SPI/I2C/HSU, firmware-specific status codes, RF activation sequencing). Implements the LLI for one specific chip. Out of scope for this document.

A change of NFC controller should require changes *only* to the Hardware/Link Layer document and its corresponding driver – never to this document or to the session/application modules. Con-

versely, a change to the application module’s command set or authorization policy should require changes *only* to the Application Module (Part II) and, where the plaintext contract changes, the Session Module (Part I) – never to this document, which never inspects plaintext content.

2 Link Layer Interface (LLI)

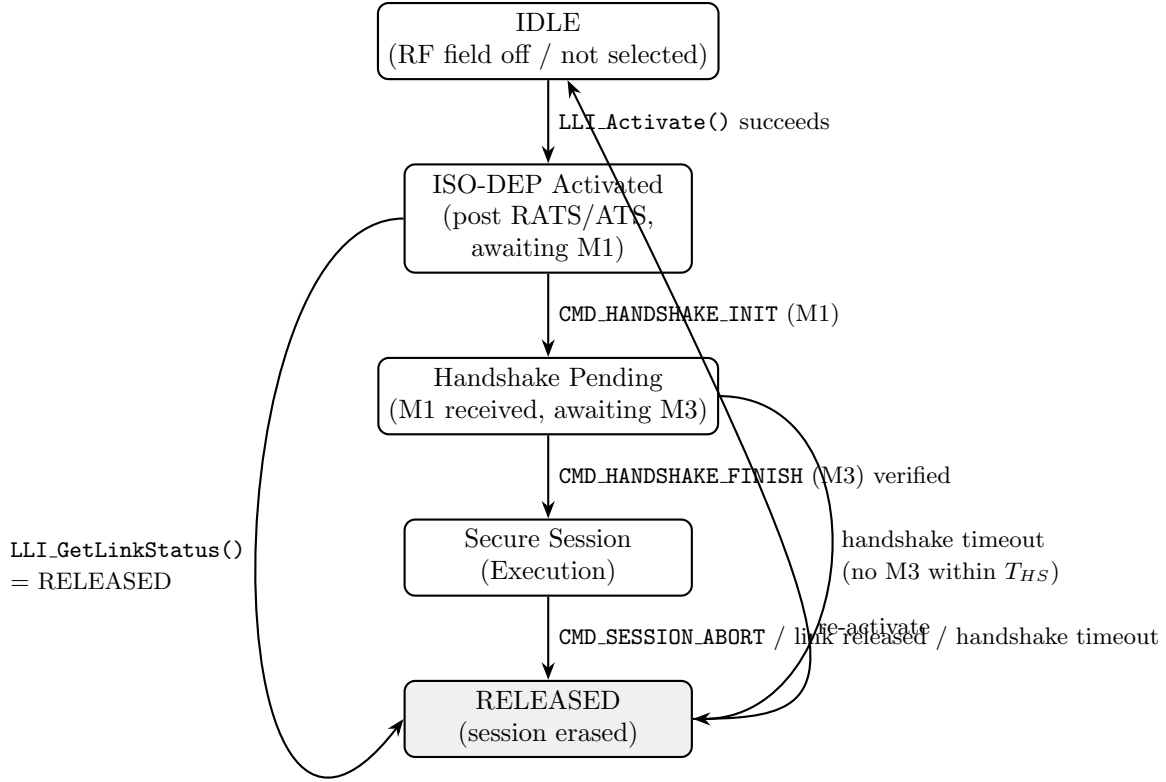
The transport layer depends on a minimal, controller-independent interface. Any conforming ISO-DEP controller driver **MUST** implement these five primitives. Exact function signatures are a Hardware/Link Layer concern; only behavior is normative here.

Primitive	Direction	Required Behavior
LLI_Activate()	Lock-internal	Performs ISO/IEC 14443-3 polling/anti-collision/selection and ISO/IEC 14443-4 RATS/ATS activation. Returns once the link is in the <i>ISO-DEP Activated</i> protocol state (Section 3) or reports failure.
LLI_ReceiveAPDU(timeout)	Target ← Initiator	Blocks (up to timeout) for the next C-APDU. Returns raw APDU bytes, or a defined error (timeout, frame-integrity error, link released).
LLI_SendAPDU(bytes)	Target → Initiator	Transmits an R-APDU. If the ISO-DEP frame-waiting-time budget is at risk of expiry due to caller processing delay, the driver is responsible for issuing ISO/IEC 14443-4 S(WTX) waiting-time-extension blocks per the standard; this is transparent to the transport layer.
LLI_GetLinkStatus()	Lock-internal	Returns one of: ACTIVE , RELEASED , ERROR . MUST reflect RF field loss or explicit de-selection promptly enough to satisfy the polling requirement in Section 8.3.
LLI_Abort()	Lock-internal	Forces immediate link teardown (deselect / halt) regardless of current protocol state.

Non-normative note: the abstract error taxonomy returned by LLI_ReceiveAPDU (timeout / frame-integrity error / link released) is intentionally coarse. A Hardware/Link Layer document **MUST** map its controller’s native error/status codes onto this taxonomy; it **MUST NOT** require the transport layer to understand controller-specific codes.

3 Target-Side Protocol State Machine

States are named in terms of ISO/IEC 14443-3/4 protocol phases plus the application states introduced by this specification. No state below implies any particular controller command.



The *Handshake Pending* intermediate state is explicit: a session entering this state that does not receive a verified M3 within the handshake timeout T_{HS} (Section 8.1) transitions directly to *RELEASED* with all ephemeral cryptographic material erased, identically to a link-loss event.

4 Data Framing and APDU Encapsulation

Framing follows ISO/IEC 7816-4 *Short APDU* (Case 4) conventions, as used generically over any ISO/IEC 14443-4 link.

4.1 Command APDU (C-APDU): Phone → Lock

Field	Size	Value / Description
CLA	1 byte	0x80 (proprietary class)
INS	1 byte	0x10 / 0x11 / 0x20 / 0x30 (Section 5)
P1	1 byte	0x00 (reserved)
P2	1 byte	0x00 (reserved)
Lc	1 byte	Length of data payload, 0x01–0xFF
Data	L_c bytes	Handshake parameters or AES-GCM secure payload

A Short APDU C-APDU (Case 4) is bounded at **261 bytes** (CLA+INS+P1+P2+Lc+255 data bytes+Le) per ISO/IEC 7816-4. This bound applies to any controller supporting Short APDU exchange and is not specific to this deployment’s hardware.

4.2 Response APDU (R-APDU): Lock → Phone

Field	Size	Value / Description
Data	0–256 bytes	Handshake response or encrypted application response
SW1	1 byte	0x90 = normal processing; see Section 9
SW2	1 byte	Status sub-code

An R-APDU is bounded at **258 bytes** (256 data bytes + SW1 + SW2), per the same Short APDU convention.

4.3 Maximum Transmission Unit and Chaining Budget

Handshake payloads fit within a single frame under any ISO-DEP controller:

Message	Payload Size	Fits in Single Frame?
M1 ($pk_P^{eph} c_P$)	64 bytes	Yes
M2 ($pk_L^{eph} c_L \text{Sig}_L$)	128 bytes	Yes
M3 (Sig_P)	64 bytes	Yes

For CMD_SECURE_PAYLOAD, the usable plaintext budget per *unchained* message:

Max Lc (Short APDU data field)	= 255 bytes
Nonce overhead	= 12 bytes
GCM Auth Tag overhead	= 16 bytes

Maximum plaintext per unchained message = 255 - 12 - 16 = 227 bytes	

Application-layer chaining is explicitly out of scope for this revision. A payload requiring more than 227 bytes of plaintext **MUST** be rejected with status word 0x6A 0x80, not silently truncated or chained. ISO/IEC 14443-4 native I-block chaining, if exposed by a given controller’s driver, **MAY** be used transparently underneath a single LLI.SendAPDU/LLI.ReceiveAPDU call in a future revision, but that mechanism is a Hardware/Link Layer concern, not a transport-layer one.

5 Instruction Set

Instruction	INS	Description
CMD_HANDSHAKE_INIT	0x10	Carries M1 ($pk_P^{eph} c_P$), 64 bytes
CMD_HANDSHAKE_FINISH	0x11	Carries M3 (Sig_P), 64 bytes
CMD_SECURE_PAYLOAD	0x20	Carries AES-256-GCM encrypted application command (nonce ciphertext tag)
CMD_SESSION_ABORT	0x30	Explicit session teardown; triggers immediate key erasure regardless of current state

CLA is 0x80 and P1/P2 are 0x00 for all four instructions.

5.1 State / Instruction Validity

Instruction	Valid State	If Invalid	Effect
CMD_HANDSHAKE_INIT	ISO-DEP Activated (awaiting M1) only	0x69 0x85	None (no state change)
CMD_HANDSHAKE_FINISH	Handshake Pending only	0x69 0x85	None if received early; 0x69 0x82 + erase if signature invalid
CMD_SECURE_PAYLOAD	Secure Session only	0x69 0x85	None (no state change)
CMD_SESSION_ABORT	Any state from ISO-DEP Activated onward	n/a (always accepted)	Immediate erase, transition to RELEASED

A repeated CMD_HANDSHAKE_INIT while already in *Handshake Pending* or *Secure Session* MUST be treated as invalid (0x69 0x85); restarting a session requires the link to be re-activated from *IDLE* via LLI_Activate().

6 Phase 1: Handshake and Key Agreement

6.1 Exchange 1: M1 → M2

Phone → Lock (CMD_HANDSHAKE_INIT): CLA 0x80, INS 0x10, Lc 0x40 (64 bytes). Data: 32 bytes pk_P^{eph} || 32 bytes c_P .

Lock → Phone (R-APDU, M2): 128 bytes data (32 bytes pk_L^{eph} || 32 bytes c_L || 64 bytes Sig_L), SW1/SW2 = 0x90 0x00.

Receipt of M1 transitions the Lock to *Handshake Pending* and starts the handshake timer T_{HS} (Section 8.1).

6.2 Exchange 2: M3 → Authentication Status

Phone → Lock (CMD_HANDSHAKE_FINISH): CLA 0x80, INS 0x11, Lc 0x40 (64 bytes). Data: 64 bytes Sig_P.

Lock → Phone (R-APDU): empty data field. SW1/SW2 = 0x90 0x00 on successful verification (transition to *Secure Session*, HKDF derivation on both sides), or 0x69 0x82 on failure (transition to *RELEASED* with immediate erase).

7 Phase 2: Secure Communication

Once in *Secure Session*, all application payloads use CMD_SECURE_PAYLOAD (0x20).

Phone → Lock: Lc = variable, up to 0xFF. Data: 12-byte nonce || ciphertext (encrypted with K_{p2e}) || 16-byte auth tag.

Lock → Phone: Data: 12-byte nonce || ciphertext (encrypted with K_{e2p}) || 16-byte auth tag. SW1/SW2 = 0x90 0x00 on success.

An AES-GCM authentication tag mismatch MUST be treated identically to a handshake signature failure: respond 0x69 0x82 and invoke the secure-erase routine immediately.

8 Timing, Liveness, and Session Guardrails

8.1 Handshake Timeout

The Lock MUST enforce a maximum duration T_{HS} between entering *Handshake Pending* (receipt of M1) and receiving a verified M3. A recommended starting value is $T_{HS} = 2\text{ s}$, comfortably above software Ed25519 signing/verification latency (single-digit milliseconds) and typical NFC round-trip overhead. This value is a transport-layer policy choice and does not depend on the underlying controller; a Hardware/Link Layer document MAY document how its controller's own frame-waiting-time (FWT) and S(WTX) behavior interacts with T_{HS} , but MUST NOT change T_{HS} itself.

8.2 Waiting Time Extension (Protocol-Level)

ISO/IEC 14443-4 defines an S(WTX) block allowing a Target to request additional processing time from the Initiator when a response cannot be produced within the negotiated Frame Waiting Time (FWT), derived from the FWI value exchanged during ATS. This specification requires only that:

- WTX handling, if needed, is issued transparently by the Hardware/Link Layer implementation of LLI_SendAPDU – the transport layer neither issues nor observes S(WTX) blocks directly.
- Application processing between LLI_ReceiveAPDU and LLI_SendAPDU SHOULD remain well under one second wherever possible, since WTX extension is bounded in practice by Initiator-side retry limits regardless of controller. Ed25519 signing on the current ESP32 target completes in low single-digit milliseconds, so this budget is not expected to be stressed by the cryptographic protocol itself.

8.3 Link Status Polling

The Lock SHOULD call LLI_GetLinkStatus() between application-level exchanges – at minimum immediately before committing any state-changing action (e.g. actuating the lock mechanism) – and MUST treat a RELEASED result identically to an explicit CMD_SESSION_ABORT. The maximum acceptable polling interval is a Hardware/Link Layer concern, since it depends on how quickly a given controller can detect RF loss; this document only requires that detection occur before any state-changing action is committed.

8.4 Secure-Erase Routine

The following trigger conditions all invoke the identical erase routine defined in the Session Module (Part I) §7 (destruction of sk^{eph} , SharedSecret, PRK, and derived session keys):

- CMD_SESSION_ABORT received in any active state
- Handshake timeout T_{HS} expiry
- LLI_GetLinkStatus() returns RELEASED
- LLI_ReceiveAPDU reports a frame-integrity error during *Handshake Pending* or *Secure Session*
- Handshake signature verification failure (M2 at Phone, or M3 at Lock)
- AES-GCM authentication tag verification failure

9 Status Words (SW1/SW2) Dictionary

SW1	SW2	Description	Trigger Condition
0x90	0x00	Success	Command executed correctly
0x6A	0x80	Incorrect parameters in data field	Invalid X25519 point (e.g. all-zero), malformed Lc, or secure payload exceeding the 227-byte unchained plaintext budget
0x69	0x82	Security status not satisfied	Handshake signature verification failed, or AES-GCM auth tag mismatch – both trigger immediate secure erase
0x69	0x85	Conditions of use not satisfied	Instruction received outside its valid state per Section 5.1
0x6A	0x81	Function not supported	Unknown INS byte

These status words are application-layer (this protocol’s own convention) and are transmitted as ordinary R-APDU trailer bytes; they do not depend on any controller-specific status/error reporting mechanism. A `LLI_ReceiveAPDU` frame-integrity error (Section 2) is a *link-layer* condition and is handled via the secure-erase routine directly – it never reaches the application as an SW1/SW2 pair, since no valid APDU was received to respond to.

10 Conformance Requirements for a Hardware/Link Layer Binding

A Hardware/Link Layer document for a specific controller MUST, at minimum:

1. Map its controller’s activation command sequence to `LLI_Activate()`, including 106 kbps Type A Passive Card Emulation configuration equivalent to what is assumed here.
2. Map its controller’s frame retrieval/transmission primitives to `LLI_ReceiveAPDU`/`LLI_SendAPDU`, including translation of controller-native error codes into the timeout / frame-integrity-error / link-released taxonomy.
3. Document how (or whether) the controller autonomously handles `S(WTX)` generation, and any bounds on that behavior (maximum single extension, maximum retries) relevant to Section 8.2.
4. Specify a concrete link-status polling mechanism and interval satisfying Section 8.3.
5. Confirm the controller supports Short APDU exchange up to the 261/258-byte bounds assumed in Section 4, or document a reduced ceiling if it does not.

11 Revision History

Version	Change	Date
0.1	Initial transport draft (APDU framing, instruction set, PN532 command sequence embedded directly); subsequently corrected malformed byte sequence and added handshake timeout, state/instruction validity table, session-abort wiring, quantified chaining budget, WTX bound, and polling guidance, still PN532-coupled at this stage	–
0.2	Removed all controller-specific content. Introduced the abstract Link Layer Interface (LLI); state machine, timing, and status words now expressed purely in ISO/IEC 14443-4 / ISO/IEC 7816-4 terms. PN532-specific material relocated to a separate Hardware/Link Layer document (now versioned 0.1). A concrete C implementation of the LLI is tracked separately in the LLI Engineering Specification (v0.3)	–
0.3	Updated references from combined <i>smart-lock_protocol_spec.pdf</i> to split the Session Module (Part I) and the Application Module (Part II). Split “Cryptographic Protocol” row in layer responsibilities table into separate Session Layer and Application Layer rows. No protocol changes.	July 24, 2026
0.4	Reframed §1.2 from a stacked layer table to a component map. Renamed “Layer Responsibilities” to “Component Map” and added Figure 3, showing the session and application modules as peers that collaborate directly with each other, not through this document. Clarified that this document’s own dependency on the Hardware/Link Layer remains a genuine stack (a real, load-bearing dependency via the LLI), unlike its relationship with the session module, which is carrier-only with no ownership in either direction. Reworded §1.1 to avoid describing this document’s own protocol-handling logic as “application logic,” since that term now refers specifically to the Application Module’s command-handling logic in a sibling document. No protocol, framing, or timing changes.	July 26, 2026

Note on numbering: this project has not yet reached a stable v1.0 release. Version numbers below 1.0 are used throughout the document set (protocol, transport, hardware/link layer, LLI) until the first frozen release, and are not directly comparable across documents – e.g. transport layer 0.3 does not imply it is “ahead of” hardware/link layer 0.1 in maturity, only that it is this document’s third drafted revision.

Part IV Hardware / Link Layer — PN532 Binding

Part Overview

This document binds the abstract Link Layer Interface (LLI) defined in *the Transport (Part III)* (Section 2) to a concrete controller: the NXP PN532 NFC front-end operating as an ISO/IEC 14443-4 Type A PICC in 106 kbps Passive Card Emulation Mode. It specifies the exact PN532 host commands, byte-level frame formats, and firmware-confirmed behaviors used to implement each LLI primitive. All PN532-specific claims below have been verified against NXP UM0701-02 (PN532 User Manual) and AN133910 (PN532 Application Note); none of this content should be assumed to generalize to a different controller. Should the Lock’s NFC front-end ever change, only this document and its corresponding driver need to be replaced.

1 Scope

This document is the sole owner of PN532-specific detail in the system. It implements, and only implements, the five LLI primitives required by the transport layer: `LLI_Activate`, `LLI_ReceiveAPDU`, `LLI_SendAPDU`, `LLI_GetLinkStatus`, `LLI_Abort`. It does not define or alter APDU framing, the instruction set, protocol state machine, timing policy (T_{HS}), or status word semantics – those remain normative in the transport layer document.

1.1 Target Configuration

Parameter	Value
Role	Target (PICC), Card Emulation Mode
Modulation	Type A, 106 kbps, Passive
Protocol	ISO/IEC 14443-4 (ISO-DEP)
Host Interface	SPI / I2C / HSU (deployment-specific; not further constrained here)

2 LLI Primitive Bindings

2.1 `LLI_Activate()` → `TgInitAsTarget` (0x8C)

The ESP32 host issues `TgInitAsTarget`, configuring the PN532 for Type A Passive Card Emulation (SENS_RES, NFCID1, SEL_RES, and ATS historical bytes are supplied as command parameters). The PN532 firmware then autonomously handles, without further host involvement:

- SENS_REQ response
- SDD_REQ anti-collision cascade
- SEL_REQ completion
- RATS reception and automatic ATS transmission using the configured historical bytes

This is confirmed firmware behavior per NXP UM0701-02 §4: “*ATS will be sent automatically to the reader which has sent a RATS.*” `LLI_Activate()` returns success once `TgInitAsTarget` completes and the ATS has been sent (i.e. the PN532 has reached its internal ISO-DEP-activated target state).

2.2 `LLI_ReceiveAPDU(timeout) → TgGetData (0x86)`

Input: `[0xD4, 0x86]` Output: `[0xD5, 0x87, Status, C-APDU (up to 262 bytes)]`

2.2.1 Status Byte Mapping

Confirmed against NXP AN133910 §3.2.2. The driver **MUST** translate these PN532-native codes into the transport layer’s abstract error taxonomy as shown:

PN532 Code	PN532 Meaning	LLI Abstract Result
0x00	Success	Valid C-APDU returned
0x01	Timeout – target has not answered	<code>TIMEOUT</code>
0x02	CRC error (contactless UART)	<code>FRAME_INTEGRITY_ERROR</code>
0x03	Parity error (contactless UART)	<code>FRAME_INTEGRITY_ERROR</code>
other	Any other non-zero status	<code>FRAME_INTEGRITY_ERROR</code> (fail closed)

Cryptographic safeguard: on any status other than 0x00, the driver **MUST** return an error to the transport layer *without* passing partial or corrupted data upward. As specified in the transport layer document, a `FRAME_INTEGRITY_ERROR` during *Handshake Pending* or *Secure Session* triggers the secure-erase routine; a corrupted frame must never reach signature verification or AES-GCM decryption.

2.3 `LLI_SendAPDU(bytes) → TgSetData (0x8E)`

Input: `[0xD4, 0x8E, DataOut[...]]` Output: `[0xD5, 0x8F, Status]`

A non-0x00 status on the `TgSetData` response indicates the R-APDU was not successfully queued for transmission; the driver **MUST** surface this as a send failure to the caller rather than assuming delivery.

2.3.1 Waiting Time Extension

Confirmed per NXP UM0701-02 §4: while acting as an ISO/IEC 14443-4 Type A PICC, the PN532 autonomously issues an `S(WTX)` request to the Initiator if the host controller does not supply the R-APDU within the negotiated Frame Waiting Time (FWT). This satisfies the transport layer’s requirement (Section 8.2 of the transport document) that WTX be handled transparently within `LLI_SendAPDU` – **no additional host logic is required** to trigger WTX on this controller.

Observed bounds on this behavior, relevant to capacity planning:

- Default maximum single WTX-extended window on PN532 deployments has been observed on the order of ~ 1 second (community-reported, configurable via RF configuration parameters governing `ATR_RES`/timeout behavior).

- Initiator-side stacks commonly cap the number of accepted WTX requests per exchange; the PN532 side does not control this limit.

Given Ed25519 signing on the ESP32 completes in low single-digit milliseconds, WTX is not expected to trigger during the handshake phase with this controller. If a future revision moves signing to the ATECC608A secure element, the added I²C transaction latency should be re-measured against this budget.

2.4 LLI.GetLinkStatus() → TgGetTargetStatus (0x8A)

The driver polls `TgGetTargetStatus` and maps its response to the LLI's three-value status:

PN532 Status	Meaning	LLI Mapping
Target activated 0x80	Link is up and selected	ACTIVE
	Target released by initiator	RELEASED
Communication error	Bus-level failure reading status	ERROR

Recommended polling interval: every 100–200 ms while in *Handshake Pending* or *Secure Session*, and additionally, synchronously, immediately before any state-changing action (e.g. before actuating the physical lock mechanism). This interval is a PN532-specific recommendation, chosen to keep detection latency for RF tear-away well below the T_{HS} handshake timeout defined by the transport layer, while avoiding excessive bus traffic. A different controller's Hardware/Link Layer document may recommend a different interval based on its own status-query cost and latency characteristics.

2.5 LLI.Abort() → InRelease / Re-issue TgInitAsTarget

Abort is implemented by issuing an `InRelease` (or equivalent deselect) sequence and returning the PN532 to a state where `TgInitAsTarget` can be re-issued to restart from *IDLE*. This is invoked by the transport layer's secure-erase routine on `CMD_SESSION_ABORT`, handshake timeout, or any detected integrity failure, so that the RF front-end and the cryptographic session state are torn down together.

3 Frame Size Conformance

The PN532's Short APDU support is confirmed per NXP UM0701-02 to satisfy the bounds assumed by the transport layer without reduction:

Direction	PN532-Confirmed Limit
External PCD → PN532 (as PICC)	Up to 261 bytes (CLA/IN-S/P1/P2/Lc + 255 data bytes + Le)
PN532 (as PICC) → PCD	Up to 258 bytes (256 data bytes + SW1/SW2)

No reduced ceiling applies; the 227-byte unchained secure-payload budget computed in the transport layer document holds unmodified for this controller. Extended APDU and Type B PICC emulation are not supported by the PN532 and are not used by this system.

4 Error and Status Code Reference

PN532 mand	Com-	Code	Notes
TgInitAsTarget		0x8C	Response 0x8D
TgGetData		0x86	Response 0x87
TgSetData		0x8E	Response 0x8F
TgGetTargetStatus		0x8A	Response 0x8B
InRelease		0x52	Response 0x53

All PN532 host commands are wrapped in the standard TFI/D4 command preamble (0xD4 for host-to-PN532, 0xD5 for PN532-to-host) per the PN532 UART/SPI/I2C host protocol; full frame checksums and preambles are defined in NXP UM0701-02 §6 and are not repeated here.

5 Revision History

Version	Change	Date
0.1	Initial extraction of PN532-specific content from the transport layer draft into a standalone Hardware/Link Layer document, structured as an explicit binding of the transport layer's Link Layer Interface (LLI)	July 26, 2026

Part V Driver Conformance Requirements

Part Overview

The four parts above (Session, Application, Transport, and the PN532 Hardware/Link Layer binding) are protocol-level specifications: they constrain *what* must happen, in terms of bytes on the wire and states in a machine, independent of any particular driver implementation. Underneath them sits a separate, code-level document set – *i2c-transport.md*, *core-driver.md*, *command-layer.md*, *api-reference.md*, and *lli.md* – that specifies the concrete ESP32/PN532 driver implementing Part IV’s bindings, plus a *communication_module_master_reference.md* that ties the whole stack together for the Session implementation effort. That code-level document set is **not** one of the four parts merged into this specification and is maintained separately in the firmware repository.

A code review of that implementation-level document set (dated July 2026) found eight concrete deltas against the as-built driver, ordered by severity: correctness bugs first, then gaps that would surface as bugs later, then hardening items safe to defer but cheap to fix now. This part records all eight as normative requirements on that lower document set and its corresponding code, so that the fixes are not lost between the firmware repository and this specification. Two of the eight (Section 2 and Section 8) were cross-checked directly against Part III and Part I of this document as part of incorporating them here; the other six (Sections 1, 3, 4, 5, 6, 7) concern driver-internal structures (`lli_config_t`, `pn532_i2c_config_t`, the `lli_init/lli_activate` call sequence) that live entirely in the code-level documents and are recorded here as the authoritative requirement the firmware must be brought into line with.

1 `lli_activate` Must Not Reference a Nonexistent Command

Severity: Bug (LLI).

lli.md describes `lli_activate` as first sending `GetGeneralStatus` (0x04) to check the external RF field flag, aborting with `LLI_ERR_INTERNAL` if a reader’s field is already present. No `pn532_get_general_status` wrapper (or any 0x04 wrapper) exists in *command-layer.md* or *api-reference.md*. As written, either `lli_activate` fails to compile, or a hand-rolled inline version duplicates command-layer logic that has never been validated.

Requirement: remove this pre-check entirely. Nothing in Part IV requires it – §2.1 of that part states that `LLI_Activate()` simply issues `TgInitAsTarget` and returns success once ATS has been sent; it does not specify a pre-flight field check, and “field already present” is not documented anywhere as a `TgInitAsTarget` failure mode. `lli_activate` MUST go directly to building `pn532_tg_init_params_t` and calling `pn532_tg_init_as_target`.

If a pre-check is genuinely wanted later (e.g. to short-circuit before a long blocking call), `pn532_get_general_status()` MUST be added to the command layer first, matching the existing wrapper style, rather than leaving `lli.c` referencing a function that does not exist.

2 `CMD_SESSION_ABORT` Must Be Honored in Every Post-Activation State

Severity: Bug (Transport).

Part III’s §5.1 is unambiguous: `CMD_SESSION_ABORT` (0x30) is valid in “any state from ISO-DEP Activated onward,” unconditionally, with effect “immediate erase, transition to `RELEASED`.” The

as-built *transport.md* narrative for the *ISO-DEP Activated (awaiting M1)* state, however, checks only for INS 0x10 and rejects everything else with 0x69 0x85 – which means a CMD_SESSION_ABORT received in that state is currently rejected and the session incorrectly *stays* in *ISO-DEP Activated* instead of transitioning to *RELEASED*.

Requirement: in the *ISO-DEP Activated* handler, the INS 0x30 check **MUST** come before the INS 0x10 check:

```
if (capdu.ins == INS_SESSION_ABORT) {
    on_erase(user_ctx);           // no-op: no keys exist yet, but
    keep it uniform
    send_rapdu(0x90, 0x00);       // best-effort
    state = TRANSPORT_STATE_RELEASED;
    return;
}
if (capdu.ins != INS_HANDSHAKE_INIT) {
    send_rapdu(0x69, 0x85);
    return;                       // stay in ISO-DEP Activated
}
```

This is the only genuine state-machine conformance bug found in the review; the *Handshake Pending* and *Secure Session* narratives in *transport.md* already special-case INS_SESSION_ABORT correctly, consistent with Part III as written in this specification. No change to Transport’s callback shape is required — a single *on_apdu/on_erase* pair remains sufficient for Session; see the discussion of that design choice within Part I.

3 lli_config_t Field Types Must Match pn532_i2c_config_t

Severity: Bug (LLI) – will not compile, or silently uses the wrong clock.

lli.md declares *lli_config_t.i2c_port* as plain *int*. The as-built *pn532_i2c_config_t* (per *api-reference.md*) declares *i2c_port_t port* (not *int*) and a separate *uint32_t clk_speed* field that *lli_config_t* has no counterpart for at all.

Requirement:

- Change *lli_config_t.i2c_port* to type *i2c_port_t* – or keep it *int* and cast explicitly at the single call site in *lli_init* where it is copied into *pn532_i2c_config_t.port*. Either is acceptable, but the choice **MUST** be made explicit and consistent; a silent implicit conversion here is exactly the kind of defect that is easy to miss in review.
- Add an *i2c_clk_hz* field (*uint32_t*, 0 = default 400 kHz) to *lli_config_t* and plumb it through to *pn532_i2c_config_t.clk_speed*. Currently no caller of *lli_init* has any way to request a non-default I2C clock, even though the driver underneath already supports it.

4 lli_init Must Validate gt_len/tk_len Up Front

Severity: Robustness (LLI).

pn532_tg_init_as_target already returns ESP_ERR_INVALID_ARG if *gt_len/tk_len* exceed 47 bytes (per *api-reference.md*’s return table), but that check only fires on the *first lli_activate* call – meaning boot succeeds, *lli_init* returns LLI_OK, and a misconfiguration (e.g. a General Bytes array copy-pasted with the wrong length) surfaces only the first time a phone taps the reader, potentially long after boot and in a context where the failure is harder to associate with its root cause.

Requirement: in `lli_init`, after copying `cfg->gt_len/cfg->tk_len` into the handle, validate both are ≤ 47 and return `LLI_ERR_INVALID_ARG` immediately if not. This is a pure fail-fast improvement: it does not change behavior for valid configurations, it only moves the failure for invalid ones from first-tap to boot time.

5 Raw-Bypass MI-Bit Chaining: Documented Assumption, Not a Silent Gap

Severity: Documented assumption (LLI) – verified safe today.

`lli_receive_apdu` deliberately bypasses `pn532_tg_get_data` (the command-layer wrapper that transparently reassembles MI-chained fragments; see *command-layer.md*’s “Chaining” note), because that wrapper collapses the PN532 status byte into `esp_err_t`, losing the granularity the LLI layer needs. That tradeoff is correct and necessary. Its side effect: LLI’s raw `pn532_send_command(0x86) + pn532_receive_response` path does *not* reassemble MI-chained fragments – if the PN532 ever set the MI bit, LLI would hand Transport only the first fragment.

Why this is safe today, not merely unexamined: Part III’s §4.3 caps a Short APDU C-APDU at 261 bytes total (CLA+INS+P1+P2+Lc+255 data+Le); the PN532’s single-frame `TgGetData` payload capacity is 262 bytes (PN532_TG_MAX_DATA_LEN, per *command-layer.md*). Since $261 < 262$, no C-APDU this protocol can ever produce will trigger MI chaining. The 227-byte unchained secure-payload budget (Part III, §4.3) makes this margin more comfortable still for `CMD_SECURE_PAYLOAD` specifically.

Requirement: not a code change – add a one-line comment at the top of `lli_receive_apdu`’s implementation stating this assumption explicitly, cross-referencing the 261/262-byte bound, so that if a future revision ever raises the plaintext budget past what fits in one PN532 frame (application-layer chaining is explicitly flagged out-of-scope-for-now in Part III, §4.3), whoever does that work is pointed straight at this function rather than discovering silent truncation via a field bug.

6 Hardware Reset Must Not Silently Lose SAM/Parameter Configuration

Severity: Robustness bug (LLI).

Two code paths can reset the physical PN532 without the LLI layer’s knowledge: (1) `pn532_i2c_write`’s bus-recovery path, which after five failed write retries calls `recover_i2c_bus()` and, if `rst_gpio` is wired, a full `pn532_i2c_reset_device()` entirely inside the I2C transport layer (per *i2c-transport.md*’s “Integration with Write Retry Loop”); and (2) a direct `pn532_reset()` call, exposed by the core driver though not currently invoked by the LLI layer.

Either path re-runs the PN532’s power-on defaults, silently discarding the `fISO14443_4_PICC`, `fAutomaticATR_RES`, and SAM mode configured once at `lli_init` time (steps 5–6 of *lli.md*’s init sequence). *core-driver.md* states directly that “the caller is responsible for re-running SAMConfiguration after reset,” but nothing is currently that caller – `lli_init` runs `pn532_sam_configuration/pn532_set_parameters` exactly once, at startup, never again. Concretely: a bus glitch mid-deployment that triggers path (1) leaves the PN532 able to answer I2C traffic again, but `TgInitAsTarget` may no longer auto-answer RATS with the configured ATS, or may not honor ISO-DEP PICC emulation at all – the Lock would appear to work (I2C alive) while being silently unreachable over NFC, a worse failure mode than an outright crash.

Requirement: re-issue `pn532_sam_configuration + pn532_set_parameters` unconditionally at the top of every `lli_activate()` call, not only once in `lli_init`. Both are idempotent, single round-trip commands (~ms each); paying that cost immediately before a call that is about to

block for seconds-to-minutes awaiting a reader tap is negligible overhead against closing this gap completely, without needing any new “was-reset” signal plumbed up from the I2C layer.

7 TgInitAsTarget Mode Flags Should Set PASSIVE_ONLY

Severity: Hardening, not a bug (LLI).

Part IV’s target configuration (§1.1) is explicitly “Type A, 106 kbps, Passive.” *lli.md* says `lli_activate` fixes Mode to `PN532_TG_MODE_PICC_ONLY` alone. Per *api-reference.md*’s mode-flag table, `PN532_TG_MODE_PASSIVE_ONLY` and `PN532_TG_MODE_PICC_ONLY` are independent bits, so setting only `PICC_ONLY` still permits an active-mode DEP activation attempt to reach far enough to be rejected later (by INS validity checks in Part III), rather than being refused at RF-activation time.

Requirement (optional, defense-in-depth): OR both bits – `PN532_TG_MODE_PASSIVE_ONLY` | `PN532_TG_MODE_PICC_ONLY` – so an active-mode or DEP activation attempt is refused by the PN532 itself, matching the “Passive only” target configuration literally rather than relying on a later layer to reject what should never have been activated. Not required for correctness, since the rest of the stack already rejects anything that is not a valid ISO-DEP C-APDU, but it is a one-flag change that makes “Passive only” load-bearing rather than aspirational.

8 Session Must Assert, Not Re-Derive, Transport’s Byte Budget

Severity: Process (Transport/Session boundary).

Part III computes `PLAINTEXT_BUDGET` = $255 - 12 - 16 = 227$ (§4.3) and rejects `Lc > 227` with `0x6A 0x80` without leaving *Secure Session*. By the time `session_handle_secure_payload` runs, `in_len` is therefore already guaranteed $\leq 227 + 28 = 255$. Session’s implementation MUST NOT re-derive or re-check this bound independently – doing so redundantly risks the two bounds drifting apart if one file changes and the other does not, with no mechanism to catch the divergence.

Requirement (documentation, not code): when Session is implemented (Part I, §6.2), have `session_handle_secure_payload` assert (debug-build only) that `in_len` ≤ 255 rather than silently truncating or re-validating against a separately maintained constant, so that a future change to Transport’s budget constant cannot quietly desync from Session’s expectations without at least tripping an assertion in testing.

9 Summary Table

#	Item	Layer	Severity	Type
1	<code>GetGeneralStatus</code> pre-check references a non-existent command (§1)	LLI	Bug	Remove the check
2	<code>CMD_SESSION_ABORT</code> not honored in <i>ISO-DEP Activated</i> (§2)	Transport	Bug	Add INS check ahead of the <code>0x10</code> check
3	<code>lli_config_t</code> field types/gaps vs. <code>pn532_i2c_config_t</code> (§3)	LLI	Bug (will not compile / silently wrong clock)	Fix type, add <code>i2c_clk_hz</code>

#	Item	Layer	Severity	Type
4	No fail-fast on oversized <code>gt/tk</code> (§4)	LLI	Robustness	Validate in <code>lli_init</code>
5	No MI-bit chaining in raw receive path (§5)	LLI	Documented assumption	Comment only, verified safe today
6	Silent hardware reset loses SAM/param config (§6)	LLI	Robustness bug	Re-apply config every <code>lli_activate</code>
7	<code>TgInitAsTarget</code> mode doesn't set <code>PASSIVE_ONLY</code> (§7)	LLI	Hardening	OR in the extra bit
8	Byte-budget duplication risk between Transport and Session (§8)	Transport/Session boundary	Process	Assert, don't re-derive

Table 8: Communication-module code-review fixes and their disposition in this specification.

10 Revision History

Version	Change	Date
0.1	Initial revision. Incorporates all eight findings of the July 2026 communication-module code review (<i>changes.md</i>) as normative requirements against the code-level driver document set (<i>i2c-transport.md</i> , <i>core-driver.md</i> , <i>command-layer.md</i> , <i>api-reference.md</i> , <i>lli.md</i>) and its corresponding firmware. Item 2 (Section 2) was cross-checked directly against Part III's state/instruction validity table as merged into this specification and confirmed already conformant there – the defect is confined to the separate <i>transport.md</i> implementation narrative, not to this specification. Item 8 (Section 8) was similarly cross-checked against Part III's payload-size derivation. Items 1, 3, 4, 5, 6, and 7 concern structures and call sequences (<code>lli_config_t</code> , <code>pn532_i2c_config_t</code> , <code>lli_init/lli_activate</code>) that exist only in the code-level document set and are recorded here as the authoritative requirement that set must be brought into line with.	July 26, 2026

Part VI Document Set History

1 Revision History (This Consolidated Document)

This section records the history of the *consolidation itself* – the act of merging four previously separate PDFs into this single specification, plus the addition of Part V. Each part’s own revision history (recording that part’s history *before* consolidation) is preserved at the end of that part and is not duplicated here.

Version	Change	Date
0.1	Initial consolidated revision. Merged <i>smartlock_session_layer.pdf</i> (v0.4), <i>smartlock_application_layer.pdf</i> (v0.2), <i>smartlock_transport_layer.pdf</i> (v0.4), and <i>smartlock_hardware_link_layer_pn532_spec.pdf</i> (v0.1) into a single document, each as its own Part with its own internal section numbering (a section-counter reset at each Part boundary preserves each source document’s original section numbers). All cross-document references of the form “see <i>smartlock_xxx.pdf</i> ” were converted to internal Part-number cross-references. All L ^A T _E X labels were namespaced per source document (ss: , app: , tx:) to eliminate label collisions introduced by the merge (fig:collab , sec:interface , and sec:revhist were each defined independently in two or more source documents). Added Part V, incorporating all eight findings of the July 2026 communication-module code review (<i>changes.md</i>) as normative requirements; see that part’s own revision history for which findings were cross-checked against this document set directly versus recorded against the separate code-level driver documents. No protocol, cryptographic, or state-machine behavior was changed by the act of consolidation itself – only cross-references, label names, and document structure.	July 26, 2026

Note on numbering: consistent with the numbering note carried in Part III, this project has not yet reached a stable v1.0 release. This consolidated document’s own version (0.1) is independent of, and not comparable to, the version number of any individual part above – e.g. this document being “0.1” does not reset or supersede Part I being independently at “0.4”; the parts retain their own version identity for traceability back to their pre-consolidation history.